Smart car with Object detection and Adaptive Speed Control

A PROJECT REPORT

Submitted by

<<Cherukuri Sri Nithya>> **<<BL.SC.U4CSE24210 >>**

<<M. Janani Sree>> **<<BL.SC.U4CSE24227 >>**

<<Saketh Aryan Amineni>> **<<BL.SC.U4CSE24245 >>**

for the course

23CSE201- Procedural Programming using C

Guided and Evaluated by

<< Priya R>>

Dept. of CSE,

AMRITA SCHOOL OF ENGINEERING, BANGALORE

AMRITA VISHWA VIDHYAPEETHAM

BANGALORE-560 035

November 2025

Smart Car with Object detection and Adaptive Speed Control

Abstract of the project

Many collisions in small autonomous vehicles occur due to the lack of intelligent obstacle detection and automatic speed control, especially in systems that rely on manual control and delayed human response.This project presents the design and implementation of a Smart car with Object Detection and Adaptive Speed Control using embedded C programming. The main objective of this project is to enhance the driving safety by automatically detecting an obstacle and adjusting the vehicle's speed based on the distance. To measure the distance between the car and an obstacle we used an Ultrasonic sensor (HC-SR04). Micro-controller changes the motor speed to slow down and eventually stop the car based on the distance to avoid accidents.

An ESP32-CAM module was used to enhance visual monitoring. It enables live video streaming of the car's front view . HC-05 Bluetooth module communicates wireless with the car, allowing manual override when needed.
This demonstrates an efficient approach towards intelligent vehicle systems. This is an important step toward safe autonomous mobility solutions.

Keywords: Ultrasonic Sensor, Adaptive Speed Control, Micro-controller, ESP32-CAM, HC-05 Bluetooth Module, Obstacle Detection, Real-time Monitoring, Collision prevention

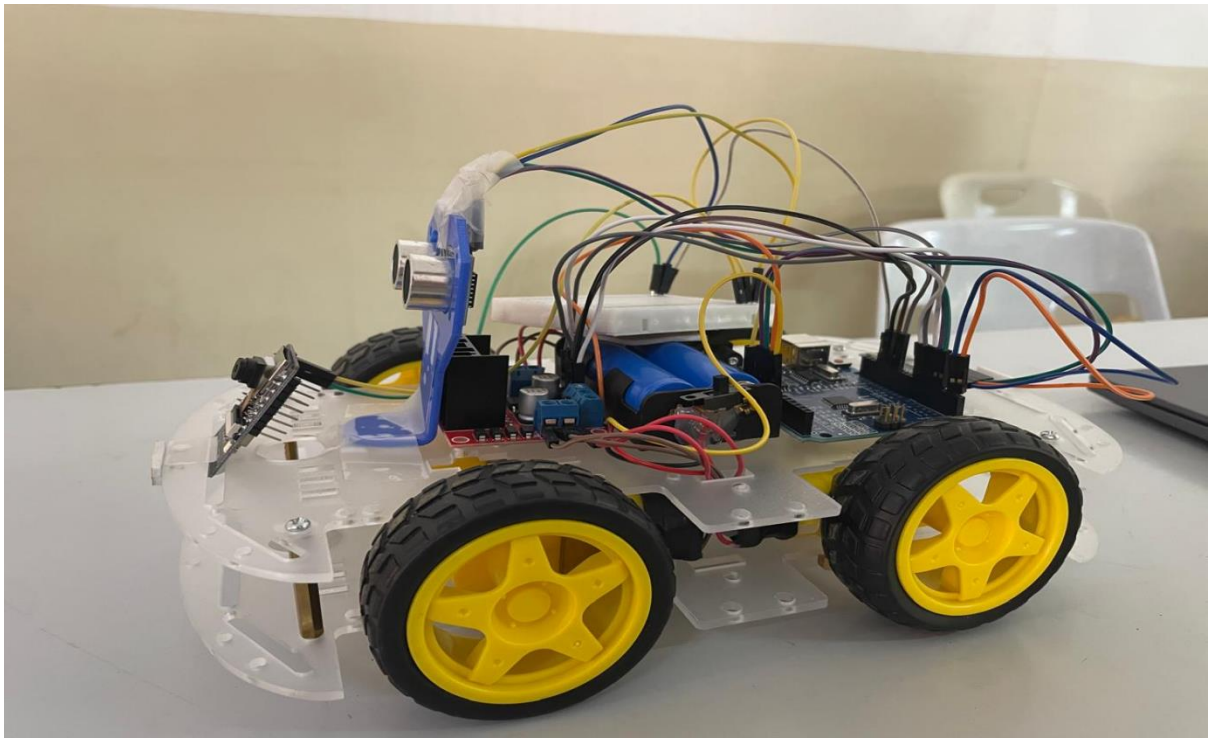
1.INTRODUCTION

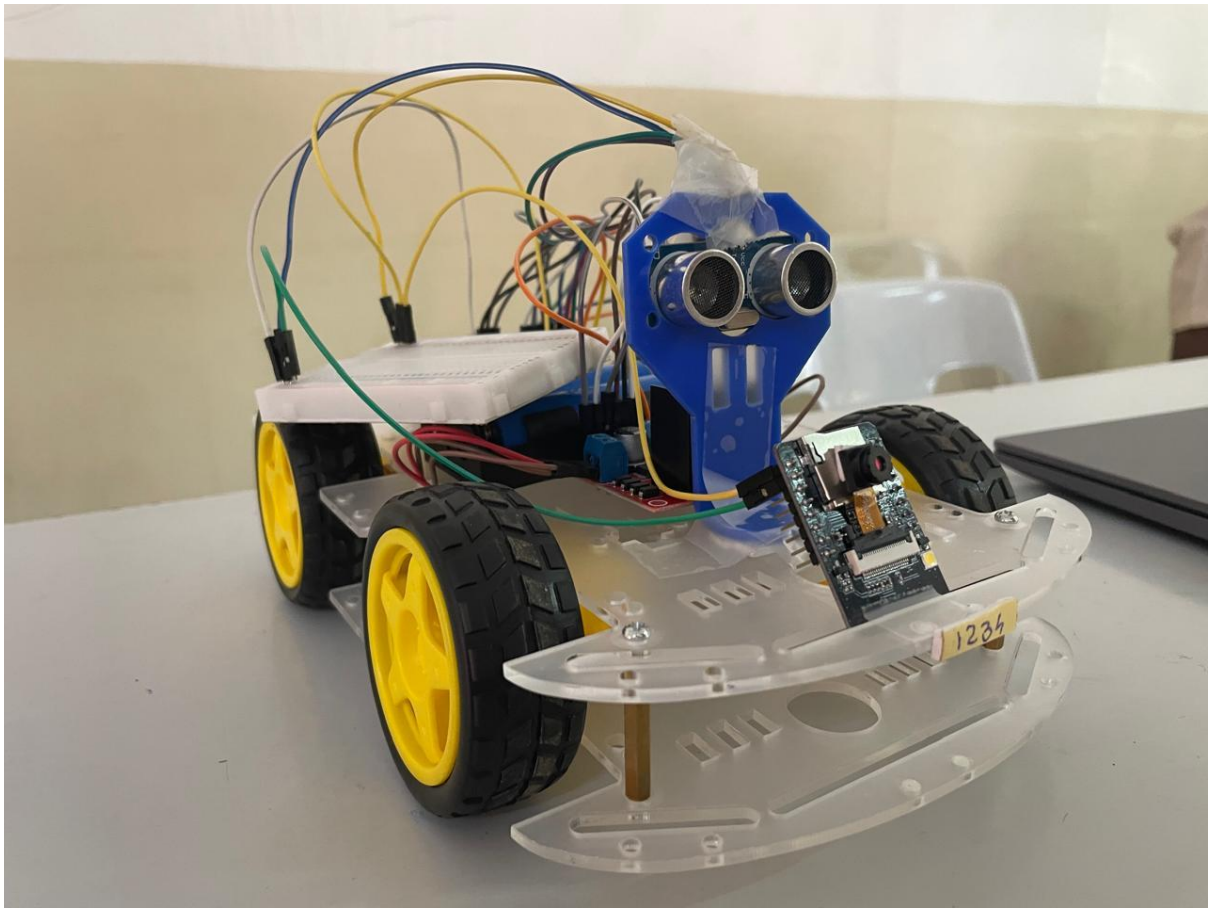
The main aim of this project is to detect obstacles and control its speed based on the distance automatically. Road accidents and Collisions in today's world, happen due to lack of slow human reaction or lack of awareness of surroundings. So, to solve this problem, we used sensors and simple programming to make a car act accordingly. An ultrasonic sensor (HC-SR04) is used to measure the distance between the car and the obstacle and reduce its speed gradually and come to a stop once it is close enough to the object. The speed reduction is done by the micro-controller on the Arduino UNO. We added an ESP32-CAM module that provides live video streaming of the car's front view, which also helps the user to see the surroundings in real time. To make the system more advance, the HC-05 Bluetooth module allows wireless communication which helps the user to control the car manually.

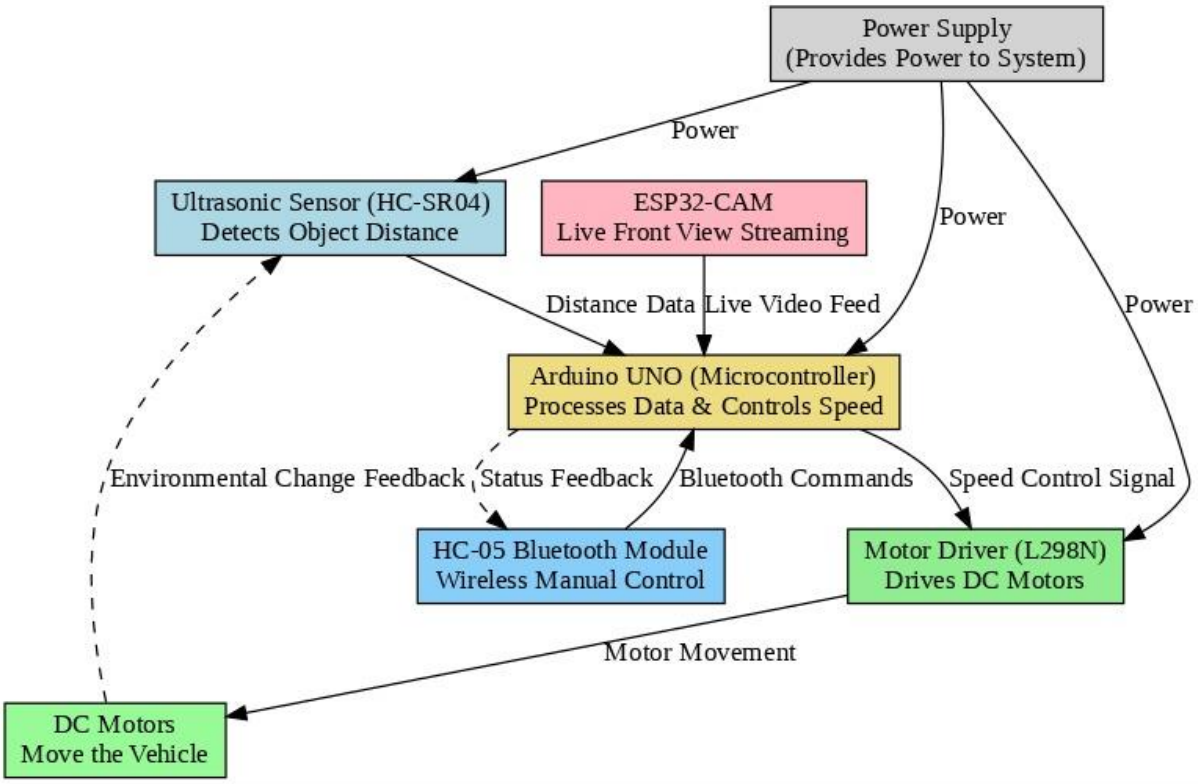
Our motivation behind this project was to use automation concepts which are used in modern intelligent vehicles. The adaptive speed control and obstacle detection is easily implemented using basic components and simple programming. Even on a small prototype scale, we wanted to create a system that shows how the technology can make driving safer and smarter.

In the future, by adding more sensors, like GPS navigation or imagine recognition, the system can be enhanced so that the car can identify objects and follow paths automatically.Thus, this project serves as a base for developing more advanced autonomous and IoT based vehice systems.

2.System Model – Diagram and Description of the diagram




1234



2.2. Description

The working of the smart car with object detection and adaptive speed control illustrates how various components detect obstacles, control the car's motion, and provide real-time visual to the user.

The input section consists of HC-SR04 ultrasonic sensor and the HC-05 Bluetooth module. The ultrasonic sensor continuously monitors the distance between the car and obstacle in front of it. The Bluetooth module allows us to manually control the car using commands such as Forward, Backward, Right, Left, Stop from a smartphone. The ESP32-CAM module additionally provides live feed of car's front view helping us to monitor the surroundings

The processing section is handled by Arduino UNO Micro-controller which receives data from both Ultrasonic Sensor and the Bluetooth Module. Using L298 Motor Driver, the speed of the car is controlled when the distance is detected. The internal program ensures that, if a obstacle is detected, within a safe distance (For Example: less that 40 cm), the car slows down.

The output section includes L298 Motor Driver, DC Motors, and the ESP32-CAM live video stream.The Motor Driver takes the control signals from the Arduino to adjust Motor direction, and speed, enabling smooth and safe navigation. For the car's motion, the DC Motors are responsible while the ESP32-CAM streams live visuals to the users device through Wi-Fi.

This model demonstrates a closed-loop intelligent vehicle system, where sensors detect obstacles, the Micro-controller processes the information, and actuators such as DC Motors and ESP32-CAM respond accordingly.

3.Implementation Details

The implementation was carried out in stages, starting with each component testing. Each module was verified separately and tested for Ultrasonic Sensor to give accurate distance readings and verifying the Motor Drivers Forward, Backward and Turn functionalities. After verifying and confirming their individual performance, all components were integrated and then tested for full system operation.

3.1. Tools and Software Used

Arduino IDE: Used for writing, compiling and uploading embedded C code to the Arduino UNO. It provides an easy interface to manage pins, libraries, and serial communication.

ESP32 Web-Interface: Configured for live video streaming from the ESP32-CAM.It also provides the user to view the car's front visuals.

Serial Monitor: It is built into Arduino IDE and this tool was used for debugging, testing sensor readings, and verifying Bluetooth communication commands.

Tinker CAD: It was used for circuit design and simulation during the early stages of development to verify the components, connection, and logic without physical hardware.

3.2. Modules and their Description

- **Input Module**

The input module of the system is the HC-SR04 Ultrasonic Sensor, which is responsible for detecting obstacles in front of the car. It emits ultrasonic waves using the trigger pin and waits for them to reflect back from an obstacle. The echo pin receives the returning wave, and the Arduino calculates the distance based on the time taken for the signal to return using the formula:

$$\text{Distance (cm)} = (\text{Duration} \times 0.034) / 2.$$

This distance data is then used by the Arduino to decide whether the car should continue moving, slow down, or stop to avoid collisions.

- **Processing Module**

The Arduino Uno microcontroller acts as the brain of the system. It receives input data from the ultrasonic sensor and Bluetooth module, processes it using the logic written in embedded C, and sends output signals to the motor driver. The Arduino generates PWM (Pulse Width Modulation) signals to control the motor speed smoothly. It also manages communication between all the modules, ensuring proper coordination between automatic and manual control modes.

- **Output / Actuation Modules**

The L298N Motor Driver serves as an interface between the Arduino and the DC motors. Since the Arduino cannot supply enough current to drive motors directly, the motor driver amplifies control signals to operate the motors. It controls both direction and speed using PWM signals from the Arduino. The DC Motors then act as the main actuators that convert electrical signals

into rotational motion, enabling the car to move forward, backward, or turn. They also respond to speed changes automatically based on the distance measured by the ultrasonic sensor.

- **Communication and Monitoring Modules**

The HC-05 Bluetooth Module enables wireless communication between the car and a smartphone. It operates using serial communication (TX and RX pins) and allows the user to send commands such as Forward (F), Backward (B), Left (L), Right (R), and Stop (S). These commands can override the automatic system, allowing manual control when needed. The ESP32-CAM Module provides a live video feed from the car's front view. It streams the surroundings over Wi-Fi, giving the user real-time visual feedback. Although the module does not record video (due to lack of SD card), it helps monitor the car's path and nearby obstacles.

- **Power Supply Module**

The Power Supply Module provides stable DC voltage to all components. A 9V or 12V battery powers the Arduino, motor driver, and other connected modules. The circuit ensures voltage regulation to prevent power fluctuations and maintain smooth operation even when the motors are under load. A reliable power source is essential for consistent performance of the entire system.

3.3. Functions Used

- **pinMode():** Sets up microcontroller pins as input or output for sensors, motors, and modules.
- **digitalWrite():** Sends a HIGH or LOW signal to control direction pins of the motor driver and trigger pin of the ultrasonic sensor.
- **analogWrite():** Generates PWM signals to control the motor speed smoothly.
- **pulseIn():** Measures the duration of the echo pulse from the ultrasonic sensor to calculate the distance from an obstacle.
- **Serial.begin(9600):** Initializes serial communication between Arduino, Bluetooth module, and the Serial Monitor for debugging.
- **delay():** Used to create small pauses between sensor readings and motor actions for stability.

User-defined Functions

- **moveForward(speed):** Moves the car forward at a specific speed.
- **moveBackward():** Moves the car in reverse direction.
- **turnLeft():** Rotates the car left.
- **turnRight():** Rotates the car right.

- **stopCar()**: Stops the motor movement completely.
- **getDistance():** Calculates the distance measured by the ultrasonic sensor.

3.4. Working of the Model

When the car runs, the Arduino repeatedly triggers the ultrasonic sensor and measures the time of the returned echo pulse. That duration is converted into a distance. If the car is commanded to move forward, the Arduino uses that distance to compute a PWM value and sends it to the motor driver enable pin; the motor driver then supplies power to the motors at that duty cycle, moving the car. If the distance becomes too small, the Arduino sets motor outputs to stop. Meanwhile, Bluetooth bytes from the smartphone arrive as UART signals; Serial.read() captures them and the Arduino changes movement mode accordingly. The camera module runs separately, streaming live video over Wi-Fi for monitoring. All communication relies on TTL-level digital signals referenced to the same ground and on carefully chosen timeouts and PWM rates to keep the system responsive and safe.

4.Code(Complete code)

4.1.Arduino-Based Bluetooth control car

```
#define IN1 13
#define IN2 12
#define IN3 11
#define IN4 7
#define ENA 5
#define ENB 6

#define TRIG 4
#define ECHO 3

char command = 'S';
unsigned long lastDistanceTime = 0;
long distance = 100;

long getDistance() {
  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);

  long duration = pulseIn(ECHO, HIGH, 20000);
  if (duration == 0) return 999;
  long dist = duration * 0.034 / 2;
  return dist;
}

void setMotorSpeed(int speed) {
  analogWrite(ENA, speed);
  analogWrite(ENB, speed);
```

```
}

void moveForward(int speed) {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  setMotorSpeed(speed);
}

void moveBackward() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  setMotorSpeed(200);
}

void turnLeft() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  setMotorSpeed(180);
}

void turnRight() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  setMotorSpeed(180);
}

void stopCar() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
  setMotorSpeed(0);
}

void setup() {
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(TRIG, OUTPUT);
```

```
  pinMode(ECHO, INPUT);
  Serial.begin(9600);
  stopCar();
}

void loop() {
  unsigned long currentTime = millis();
  if (currentTime - lastDistanceTime >= 60) {
    distance = getDistance();
    lastDistanceTime = currentTime;
  }

  if (Serial.available() > 0) {
    command = Serial.read();
  }

  switch (command) {
    case 'F': {
      if (distance > 40 && distance < 200) {
        int speed = map(distance, 40, 200, 0, 255);
        speed = constrain(speed, 0, 255);
        moveForward(speed);
      }
      else if (distance <= 40) {
        stopCar();
        Serial.println("□ Obstacle detected! Car stopped.");
      }
      else {
        moveForward(255);
      }
      break;
    }

    case 'B':
      moveBackward();
      break;

    case 'L':
      turnLeft();
      break;

    case 'R':
      turnRight();
      break;

    case 'S':
      stopCar();
      break;
  }
}
```

4.2. ESP32-CAM video streaming code

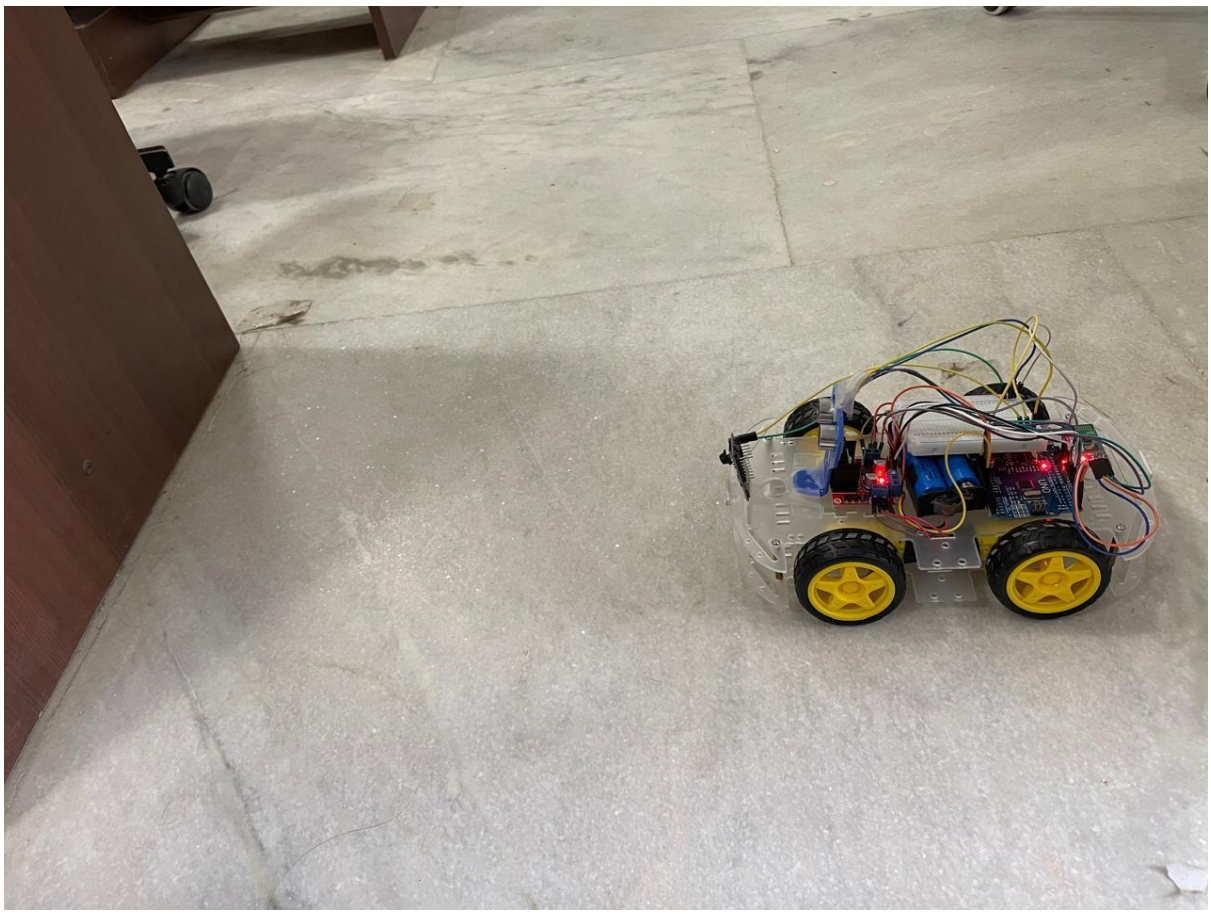
```
#include "esp_camera.h"
#include <WiFi.h>
#include "board_config.h"

const char *ssid = "realme";
const char *password = "8904407646";

void startCameraServer();
void setupLedFlash();

void setup() {
  Serial.begin(115200);
  Serial.setDebugOutput(true);
  Serial.println();

  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sccb_sda = SIOD_GPIO_NUM;
  config.pin_sccb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.frame_size = FRAMESIZE_UXGA;
  config.pixel_format = PIXFORMAT_JPEG;
  config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
  config.fb_location = CAMERA_FB_IN_PSRAM;
  config.jpeg_quality = 12;
  config.fb_count = 1;

  if (config.pixel_format == PIXFORMAT_JPEG) {
    if (psramFound()) {
      config.jpeg_quality = 10;
      config.fb_count = 2;
      config.grab_mode = CAMERA_GRAB_LATEST;
```

```
    } else {
      config.frame_size = FRAMESIZE_SVGA;
      config.fb_location = CAMERA_FB_IN_DRAM;
    }
  } else {
    config.frame_size = FRAMESIZE_240X240;
#if CONFIG_IDF_TARGET_ESP32S3
    config.fb_count = 2;
#endif
  }

#if defined(CAMERA_MODEL_ESP_EYE)
  pinMode(13, INPUT_PULLUP);
  pinMode(14, INPUT_PULLUP);
#endif

  esp_err_t err = esp_camera_init(&config);
  if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
  }

  sensor_t *s = esp_camera_sensor_get();
  if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1);
    s->set_brightness(s, 1);
    s->set_saturation(s, -2);
  }
  if (config.pixel_format == PIXFORMAT_JPEG) {
    s->set_framesize(s, FRAMESIZE_QVGA);
  }

#if                    defined(CAMERA_MODEL_M5STACK_WIDE)                    ||
defined(CAMERA_MODEL_M5STACK_ESP32CAM)
  s->set_vflip(s, 1);
  s->set_hmirror(s, 1);
#endif

#if defined(CAMERA_MODEL_ESP32S3_EYE)
  s->set_vflip(s, 1);
#endif

#if defined(LED_GPIO_NUM)
  setupLedFlash();
#endif

  WiFi.begin(ssid, password);
  WiFi.setSleep(false);

  Serial.print("WiFi connecting");
```

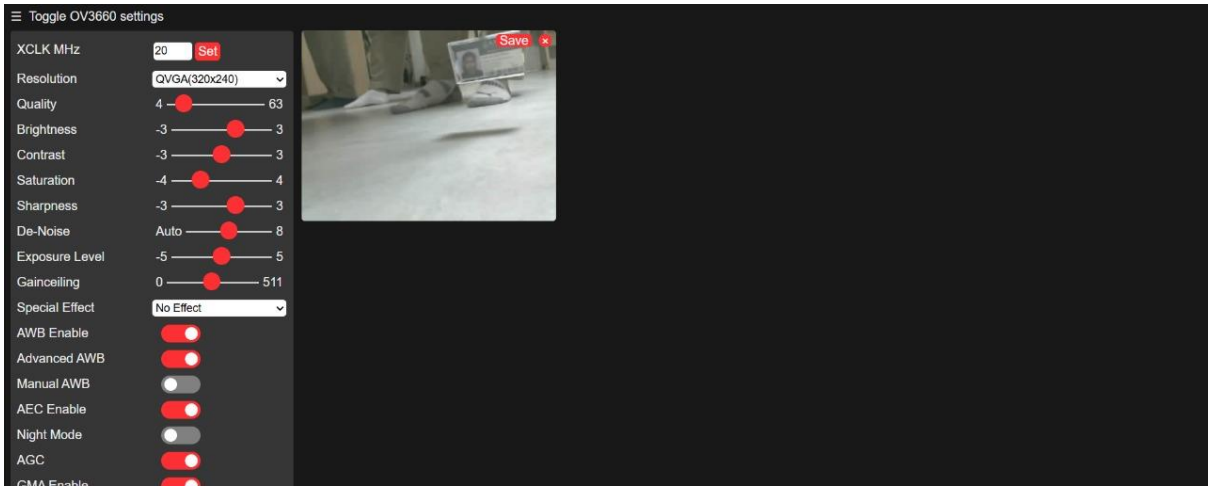
```
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");
  startCameraServer();
  Serial.print("Camera Ready! Use 'http://");
  Serial.print(WiFi.localIP());
  Serial.println("' to connect");
}
void loop() {
  delay(10000);
}
```

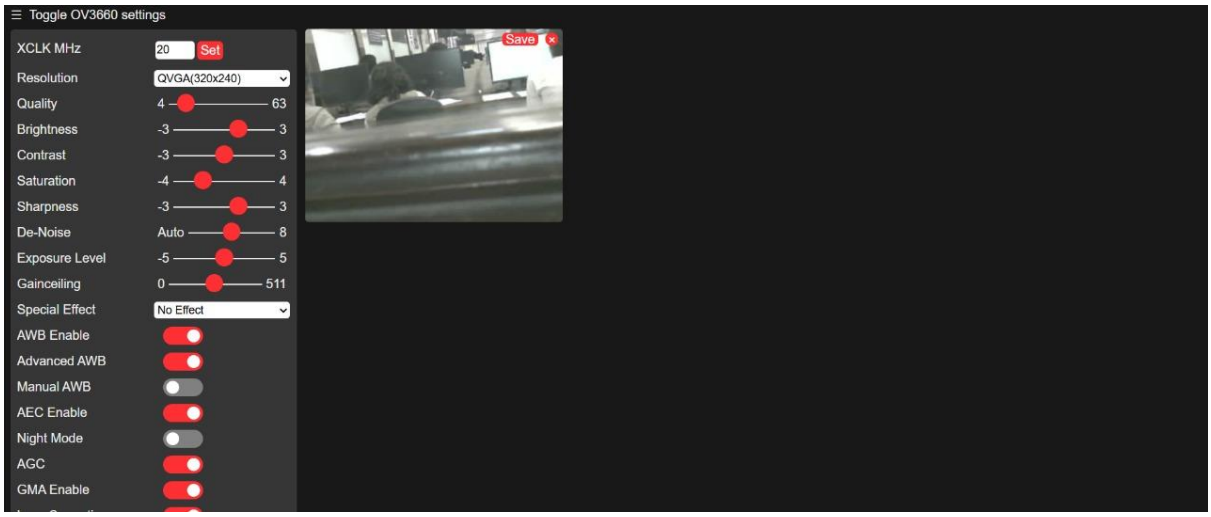
5. Output

5.1. Object Detection and Adaptive Speed Control



5.2. ESP32-CAM Live Streaming Screenshots





6.Conclusion

This project demonstrates a simple yet effective approach to automation through the development of a smart car capable of detecting obstacles and adjusting its speed automatically. By combining sensor data processing, motor control, and wireless communication, the system ensures safer and smarter navigation. The addition of a live camera feed enhances real-time monitoring, making the setup both practical and interactive. Overall, this project reflects how embedded systems and automation can work together to build intelligent and safety-oriented vehicle solutions.

7.References

- https://randomnerdtutorials.com/esp32-cam-video-streaming-web-server-camera-home-assistant/
- https://lastminuteengineers.com/l298n-dc-stepper-driver-arduino-tutorial/
- https://projecthub.arduino.cc/lakshyajhalani56/l298n-motor-driver-arduino-motors-motor-driver-l298n-7e1b3b
- https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf
- https://projecthub.arduino.cc/Fouad_Roboticist/dc-motors-control-using-arduino-pwm-with-l298n-h-bridge-25b3b3

- https://components101.com/sensors/ultrasonic-sensor-working-pinout-datasheet
- https://components101.com/sites/default/files/component_datasheet/HC-05%20Datasheet.pdf